



PDE

POLICY DEPLOYMENT ENGINE

- Aaditya Kulkarni
- Student Id: 223309593
- Service: Secret Manager

Table of Contents

1. Service Overview	3
2. Resource Types.....	3
2.1.1 Policy: Annotations Required	3
2.1.2 Policy: CMEK Required	3
2.1.3 Policy: Destroy TTL Required	3
2.1.4 Policy: Labels Required	3
2.1.5 Policy: Region Redundancy Enforced.....	4
2.1.6 Policy: Rotation and TTL Enforced.....	4
2.1.7 Policy: Topic Attachment Required	4
2.1.8 Policy: TTL Enforced	4
2.2.1 Policy: IAM Conditions Enforced.....	4
2.2.2 Policy: Complex IAM Conditions Enforced.....	4
2.2.3 Policy: IAM Role Restrictions	4
2.2.4 Policy: Public Access Blocked	4
2.3.1 Policy: Base64 Encoding Required.....	4
2.3.2 Policy: Deletion Policy Whitelisted	5
2.3.2 Policy: Write-Only Data Flag Enforced.....	5
2.3.3 Policy: Rotation Period Required	5
3. Suggestions for Future Development	5

1. Service Overview

Google Cloud Secret Manager provides a secure and centralized way to manage secrets such as API keys, passwords, and certificates. It ensures sensitive data is protected through access controls, encryption, versioning, and auditing features that are critical for modern cloud-based security.

2. Resource Types

This policy suite applies to the following Secret Manager resource types:

- `google_secret_manager_secret`: The top-level secret container.
- `google_secret_manager_secret_iam_binding`: IAM bindings for who can access or manage a secret.
- `google_secret_manager_secret_version`: The actual data/version of a secret.

Each resource serves a unique purpose in controlling, storing, and auditing sensitive credentials.

2.1.1 Policy: Annotations Required

This policy ensures that metadata annotations like `created_by` and `env` are added to secrets. It helps with traceability, auditing, and environment identification during reviews or investigations.

2.1.2 Policy: CMEK Required

This policy enforces that secrets use a customer-managed encryption key (CMEK) instead of default Google-managed encryption. It improves compliance and gives organizations more control over cryptographic protection.

2.1.3 Policy: Destroy TTL Required

This policy checks that the `version_destroy_ttl` attribute is set. It ensures secrets aren't deleted immediately upon destroy and provides a buffer for accidental deletions.

2.1.4 Policy: Labels Required

This policy enforces that at least one label is set for each secret. Labels help categorize secrets by team, purpose, or environment for easier governance and filtering.

2.1.5 Policy: Region Redundancy Enforced

This policy ensures that user-managed replication includes at least two regions. It protects against regional outages and ensures high availability of secrets.

2.1.6 Policy: Rotation and TTL Enforced

This policy requires that secrets have both rotation and TTL configurations. This helps reduce the risk of stale secrets being left active indefinitely.

2.1.7 Policy: Topic Attachment Required

This policy checks that at least one Pub/Sub topic is attached to secrets. This enables alerting or auditing workflows when changes occur.

2.1.8 Policy: TTL Enforced

This policy enforces that a TTL is specified for each secret. TTL ensures secrets are automatically expired, which helps with secret hygiene and lifecycle management.

2.2.1 Policy: IAM Conditions Enforced

This policy requires that IAM bindings use condition blocks. Conditions can restrict access using expiry dates or logical expressions, which prevents long-term overprivileged access.

2.2.2 Policy: Complex IAM Conditions Enforced

This policy enforces more advanced IAM conditions, including both expiry timestamps and weekday restrictions. It helps in tightly controlling access windows.

2.2.3 Policy: IAM Role Restrictions

This policy blocks the use of overly permissive roles like owner, editor, or viewer. It ensures only fine-grained roles like secretAccessor are used to follow the principle of least privilege.

2.2.4 Policy: Public Access Blocked

This policy prevents adding public members such as allUsers or allAuthenticatedUsers to secrets. It ensures secrets are not exposed unintentionally.

2.3.1 Policy: Base64 Encoding Required

This policy enforces that binary-like secret data is base64 encoded using `is_secret_data_base64 = true`. This helps maintain data integrity and avoids decoding errors.

2.3.2 Policy: Deletion Policy Whitelisted

This policy ensures `deletion_policy` is set to `DISABLE` or `ABANDON` instead of `DELETE`. It protects against permanent loss by allowing recovery time before deletion.

2.3.2 Policy: Write-Only Data Flag Enforced

This policy ensures sensitive data is stored using write-only fields (`secret_data_wo`). It prevents secrets from being exposed in plaintext via Terraform state or plan output.

2.3.3 Policy: Rotation Period Required

This policy enforces the presence of `rotation_period` to ensure secrets are rotated periodically. It helps minimize the window of exposure in case of leaks.

3. Suggestions for Future Development

- Require naming conventions for secrets to match project or environment tags.
- Detect secrets that haven't been accessed in over 90 days for cleanup.
- Enforce specific key algorithms in CMEK policies (e.g., RSA vs. AES).
- Validate consistency between secret labels and their replication strategy.
- Flag secrets shared across multiple projects unless explicitly approved.
- Audit secrets with IAM bindings that override org-level restrictions.
- Enforce minimum TTL durations to prevent secrets from expiring too quickly.
- Require owner annotations to identify who is accountable for each secret.
- Enforce a 90-day maximum age for unrotated secrets.